SQL avancé

SUJET DU PROJET

Séances 4 à 6 · Votre projet

Matière	SQL avancé · Ycode A2627_4728
Année scolaire	2026-2027
Formateur	Luc Charlopeau
Groupe cible	Bachelor 2 Informatique, IA & Data et Cybersécurité
Dernière mise à jour	04/10/2026

SQL avancé · Bachelor 2 · Ycode A2627_4728
Projet de groupe · en classe lundi 12 et mardi 13 octobre · présentations jeudi 15 · rendu le vendredi 16 octobre 2026 à 23 h 59

La semaine dernière, vous avez vu chaque notion sur la base du festival. Cette semaine, vous les appliquez à **votre** base, sur un sujet que vous choisissez. Puis vous m'envoyez le lien de votre dépôt.

En bref

- Un groupe de **3 ou 4**.
- Un **sujet libre**, sur **PostgreSQL**, avec au moins **quatre tables reliées entre elles**.
- Sur cette base, vous ajoutez les **cinq choses vues en cours** (le tableau plus bas).
- Vous rendez un **dépôt GitHub public** : un script SQL et un `README.md`.

Ce qui est noté, c'est l'application des notions du cours. Le sujet lui-même n'est pas noté : prenez-en un qui vous amuse.

Des idées de sujets

Piochez, adaptez, ou inventez le vôtre. Une base que vous avez déjà (NexusOps, un projet personnel) convient aussi. Seule la billetterie de festival est exclue : c'était celle du cours.

Idée	Quatre tables possibles	Une règle à faire respecter
Ligue de foot amateur	équipes, joueurs, matchs, buts	pas de but pour un joueur qui n'est pas sur la feuille de match
Jeu de cartes à collectionner	joueurs, cartes, collections, échanges	on n'échange pas une carte qu'on ne possède pas
Streaming musical	artistes, titres, utilisateurs, écoutes	un compte gratuit a un nombre d'écoutes limité par jour
Revente entre particuliers	membres, annonces, achats, avis	pas d'avis sans achat
Salle de sport	adhérents, abonnements, cours, réservations	pas plus de réservations que de places dans le cours
Serveur communautaire	membres, salons, messages, sanctions	un membre banni ne poste plus
Bar à jeux de société	jeux, tables, réservations, emprunts	un jeu déjà emprunté ne s'emprunte pas
Covoiturage	conducteurs, trajets, passagers, réservations	pas plus de passagers que de sièges
Location de trottinettes	trottinettes, stations, utilisateurs, trajets	une trottinette en panne ne se loue pas
Bibliothèque de mangas	séries, tomes, lecteurs, emprunts	trois emprunts en cours au maximum
Tournoi e-sport	équipes, joueurs, matchs, résultats	un résultat validé ne se modifie plus
Livraison de repas	restaurants, plats, commandes, livreurs	une commande livrée ne s'annule plus

---

Ce que votre base doit contenir

Élément	Minimum	Votre modèle dans le cours
Une fonction ou une procédure	une	séance 1 : `billets_client`, `rembourser`
Un trigger	un	séance 2 : la survente, le double scan, le journal
Des index	deux, chacun mesuré avant et après	séance 2 : l'exercice sur les index
Des rôles	deux rôles-métiers aux droits différents, avec un utilisateur chacun	séance 3 : le guichet et la compta
Des vues	une par rôle ; le rôle a le droit sur sa vue, pas sur les tables	séance 3 : la vue du guichet
Des données	au moins 10 000 lignes dans votre plus grosse table	la recette plus bas

Choisissez vos deux profils d'utilisateurs dès le début : par exemple quelqu'un qui fait le travail courant, et quelqu'un qui contrôle sans rien modifier.

Ce que vous rendez

Un **dépôt GitHub public** avec deux fichiers :

Fichier	Contenu
`projet.sql`	tout votre SQL, dans cet ordre : les tables, les données, la fonction ou la procédure, le trigger, les vues, les rôles, les index
`README.md`	votre sujet en deux lignes, puis quelques lignes par élément (voir ci-dessous)

`projet.sql` doit s'exécuter **sans erreur sur une base vide**. Testez-le avant de rendre.

Dans le `README.md`, pour chaque élément du tableau, écrivez deux ou trois lignes :

> **Ce qu'il fait** : …
> **Pourquoi il est là** : … (pour le trigger : pourquoi dans la base plutôt que dans l'application)
> **La preuve** : la requête qui le montre, et ce qu'on observe.

Pour les index, la preuve est le temps avant et le temps après. Pour les rôles, c'est une action autorisée et une action refusée. On ne recopie pas le code : on dit ce qu'il fait et comment on le vérifie.

Chaque membre commite son propre travail : l'historique Git montre qui a fait quoi.

Le déroulé

Chaque séance a lieu deux fois : le matin (8 h 45 à 12 h 45), **groupe 1** (CYBERSEC B2 + INFOR B2) ; l'après-midi (13 h 45 à 17 h 45), **groupe 2** (IADAT B2 + INFOR B2 B). Elle se termine un quart d'heure avant l'heure.

Séance 4 · lundi 12 octobre

Contenu	Durée
Lancement : le sujet, ce qu'il faut rendre	10 min
Les tables et les données	85 min
Pause	30 min
La fonction ou la procédure, puis le trigger	80 min
Point d'étape : chaque groupe montre ce qui tourne	20 min
Marge : fin de la séance	15 min

Séance 5 · mardi 13 octobre

Contenu	Durée
Lancement : où en est chaque groupe	5 min
Les rôles et les vues	90 min
Pause	30 min
Les index	45 min
Le `README.md`, et le script relancé sur une base vide	40 min
Point d'étape : ce qui reste à faire avant jeudi	15 min
Marge : fin de la séance	15 min

Mercredi 14, pas de cours : c'est du temps pour finir.

Séance 6 · jeudi 15 octobre

Contenu	Durée
Lancement : l'ordre de passage	10 min
Finition : le script sur une base vide, la répétition de la démo	35 min
Présentations, première moitié des groupes	60 min
Pause	30 min
Présentations, suite	90 min
Marge : questions, rappel du rendu, fin de la séance	15 min

Deux recettes

Lancer votre script sur une base vide

Vous pouvez garder le conteneur du festival : on y crée une seconde base, vide, pour votre projet. Depuis le dossier `base-festival`, avec le chemin de votre fichier :

```
docker compose exec db psql -U festival -c "DROP DATABASE IF EXISTS projet WITH (FORCE)" -c
        "CREATE DATABASE projet"
docker compose cp ../../mon-projet/projet.sql db:/tmp/projet.sql
docker compose exec db psql -U festival -d projet -f /tmp/projet.sql
```

La première ligne repart d'une base vide. Relancez les trois à chaque fois que vous voulez tout rejouer. Pour vous y promener ensuite : `docker compose exec db psql -U festival -d projet`.

Les rôles existent pour tout le serveur, pas seulement pour une base : au deuxième passage, `CREATE ROLE` affiche `already exists`. C'est sans gravité.

Générer 10 000 lignes

Personne ne tape 10 000 lignes à la main. `generate_series` fabrique des lignes numérotées, et on calcule chaque colonne à partir du numéro `i`. Exemple, pour une table de matchs reliée à 20 équipes :

```sql
INSERT INTO matchs (equipe_id, score, joue_le)
SELECT 1 + (i % 20),                    -- une équipe parmi les 20
       (random() * 5)::int,             -- un score entre 0 et 5
       date '2026-01-01' + (i % 300)    -- une date parmi 300 jours
FROM generate_series(1, 10000) AS i;
```

Outil	Rôle
`generate_series(1, 10000) AS i`	10 000 lignes, numérotées de 1 à 10 000
`1 + (i % 20)`	un nombre de 1 à 20 : pratique pour une clé étrangère vers une table de 20 lignes
`(random() * 5)::int`	un entier au hasard entre 0 et 5
`date '2026-01-01' + (i % 300)`	une date
`'Équipe ' || i`	un texte : `Équipe 1`, `Équipe 2`…

Une seule table a besoin de volume. Les autres peuvent contenir quelques lignes tapées à la main. Remplissez d'abord les tables référencées, puis celles qui pointent vers elles.

La présentation du jeudi 15

Chaque groupe passe à son tour, sur son créneau habituel : **8 minutes de démo, 4 minutes de questions**. L'ordre de passage est donné en début de séance.

La démo suit toujours la même trame, en direct sur votre base :

1. **Un rôle** : `SET ROLE` sur un utilisateur, une action autorisée, une action refusée.
2. **Une vue** : ce qu'elle montre, et ce qu'elle cache.
3. **Un index** : la requête, le temps avant, le temps après.
4. **Un trigger** : l'action qui le déclenche, et ce qu'on observe.

Pas de diaporama : on montre la base qui tourne. Préparez vos requêtes dans un fichier, pour ne pas les taper devant tout le monde.

Pendant les questions, l'enseignant interroge un membre au hasard : chacun doit pouvoir expliquer n'importe quelle partie du projet. La démo ne rapporte pas de points à part : elle sert à constater que tout fonctionne.

Le rendu

Déposez sur Moodle un fichier `.txt` qui contient **le lien de votre dépôt GitHub** et **le prénom et le nom de chaque membre du groupe**. Un seul dépôt par groupe, déposé par un membre.

Au plus tard le vendredi 16 octobre 2026 à 23 h 59. C'est le dernier commit avant cette heure qui est corrigé : vous avez encore le temps de finir après votre présentation. Vérifiez que le dépôt est bien **public** : un lien qui ne s'ouvre pas ne peut pas être corrigé.

La grille

Critère	Points
DCL, rôles et privilèges	5
Vues et abstraction	4
Index et justification performance	4
Fonctions, procédures, triggers	5
Qualité du script	2
Total	**20**

Les erreurs qui coûtent le plus de points :

Critère	Erreur fréquente
Rôles	un seul rôle qui a tout ; des droits donnés aux personnes plutôt qu'aux rôles-métiers ; aucun test
Vues	le profil a aussi le droit sur la table : la vue ne protège plus rien
Index	aucun temps avant et après ; ou trop peu de lignes pour voir une différence
Trigger	un trigger qui ne protège rien, ou un message d'erreur qui n'explique pas le refus
Script	il échoue sur une base vide : des objets créés dans le mauvais ordre